CNS1001 Project 2 Proposal

Darien Henry

Tean Thomas

Domanick McFarlane

Student ID#s: 2205663, 2601597, 2602975

University of Technology, Jamaica

School of Computing and Information Technology

Contemporary Programming

CNS1001

Mr. Wayne Thompson

November 10, 2026

Table of Contents

Table of Contents **2**
1. Problem Definition **3**
2. Program Description **4**
Main Features 4
Inputs 5
Outputs 6
3. Programming Concepts to be Used **6**
4. Group Member Roles **8**
Darien Henry — Data & Business Logic 8
Domanick McFarlane — GUI & Validation 8
Tean Thomas — Reports, Testing & Diagrams 9

1. Problem Definition

Paper logbooks or ad hoc spreadsheets are commonly used by small and mid-sized hotels in Jamaica to handle reservations, room availability, guest records, and staff assignments. These techniques result in:

1. Double-bookings — two guests assigned to the same room because availability is not checked centrally.
2. Lost revenue — staff forget to record bookings or miscalculate the cost of a stay.
3. Poor maintenance tracking — rooms that need repair are still offered to guests.
4. No reliable records — guest history, staff roles, and revenue reports cannot be generated quickly.

Why is it worthwhile to solve? A straightforward desktop reservation system generates immediate reports, centralises these records, and enforces validation (e.g., cannot book an occupied room). This directly lowers booking errors and provides managers with revenue and occupancy data, which is a significant operational gain for a small hotel.

The intended users are hotel managers (reports), housekeeping/maintenance personnel (room status), and front desk receptionists (day-to-day bookings).

2. Program Description

The Hotel Reservation Management System (HRS) is a desktop Python program that uses a Tkinter graphical user interface. Hotel employees can use it to log in, manage rooms, visitors, employees, and reservations, as well as create reports. To ensure that records are preserved between sessions, all data is saved to a local file.

Main Features

1. Secure Login — up to 3 attempts using stored credentials.

2. Dashboard — sidebar navigation with live tables showing rooms, guests, bookings, and staff.

3. Room Management — add rooms with auto-generated IDs (R001, R002…), type (Single/Double/Suite), nightly rate, and capacity.

4. Guest Management — add/update guests with auto-generated IDs (G001…), contact details, and ID number.

5. Staff Management — add staff with auto-generated IDs (E001…), role (Receptionist/Housekeeping/Manager), and shift.

6. Room Status Updates — change a room between Available, Occupied, and Under Maintenance.

7. Create Booking — select a guest, select an available room, enter check-in and check-out dates, and the system calculates the total cost (nights × nightly rate). The room is then marked Occupied.

8. Check-Out — closes the active booking, records the final bill, and frees the room.

9. Maintenance Logging — mark a room Under Maintenance with a reason and date.

10. Reports — occupancy summary, revenue report, and a full bookings list, with CSV export for external use.

Program Flow

Start → Login (3 attempts) → Dashboard

→ User selects a module from sidebar

→ Module opens (form or table view)

→ User enters data

→ Validation runs (rejects bad input, shows error)

→ Valid data saved (updates in-memory structures + JSON file)

→ Dashboard tables refresh

→ User logs out or exits

Inputs

• Login credentials

• Room details (type, rate, capacity)

• Guest details (name, contact, ID number)

• Staff details (name, role, shift)

• Booking details (guest, room, check-in, check-out)

• Maintenance details (room, reason, date)

Outputs

• Live dashboard tables

• Confirmation/error popups

• Calculated booking cost

• Printed/saved reports (occupancy, revenue, bookings)

• CSV export files

• Persisted JSON data file

3. Programming Concepts that will be Used

Concept	**How it is used in HRS**
Functions / main()	main() initializes the app; at least 15 user-defined functions handle login, room/guest/staff management, bookings, check-out, maintenance, reporting, validation helpers, and file I/O.
Loops	while loop for login attempts (max 3); for loops to refresh dashboard tables and iterate bookings for reports.
Conditionals	Availability checks before booking; role validation; date validation (check-out > check-in); rate must be positive; capacity must be positive integer.

Data structures	Lists — rooms, guests, bookings, staff, each holding dictionaries. **Dictionaries** — individual records ({"room_id": "R001", "rate": 150.0, ...}) and lookup maps (e.g., room_index = {"R001": 0, ...} for fast status updates). A **set** may be used for valid room types.
Input validation	Every field validated: empty strings rejected; numeric fields checked with try/except; dates parsed with datetime.strptime; room rate > 0; capacity is a positive integer; booking requires an Available room; check-out must be after check-in.
try/except	Wraps all numeric conversions (float, int), date parsing, JSON file reads/writes, and CSV exports so bad input or a corrupt file never crashes the program.
File handling	JSON (hotel_data.json) for full persistence of all lists, saved next to the executable/script for portability. CSV (bookings_report.csv, occupancy_report.csv) for report exports — the optional-but-rewarded file type from the brief.
GUI (Tkinter)	Login window → main dashboard (sidebar + Treeview tables) → modular popup forms for each operation.

4. Group Member Roles

Each member owns distinct functions and modules, ensuring every member has:

- ❖ GitHub commits,
- ❖ At least one module they can fully explain in the presentation,
- ❖ Testing and documentation responsibilities.

Darien Henry — Data & Business Logic

1. models.py equivalent section: room/guest/staff/booking record structures
2. load_data() and save_data() (path-safe JSON persistence)
3. calculate_booking_cost() — nights × rate
4. create_booking() and check_out() logic
5. update_room_status() logic
6. Testing: booking flow, cost calculation, check-out
7. Documentation: data model section of README

Domanick McFarlane — GUI & Validation

1. Login window with 3-attempt loop
2. Dashboard layout (sidebar, Treeview tables)
3. Popup forms for every module (Add Room, Add Guest, Add Staff, Booking, Maintenance)
4. All input validation helpers (empty, numeric, date, availability)
5. Flowcharts for login, booking, and check-out flows

6. Testing: login, all input validation cases
7. Documentation: GUI + validation section of README

Tean Thomas — Reports, Testing & Diagrams

1. Report generation (occupancy, revenue, bookings)
2. CSV export functions
3. Hierarchy chart and IPO charts for each module
4. Pseudocode for all major functions
5. Test case log (5+ cases with evidence) and README testing section
6. Documentation: README assembly, AI disclosure, contribution report

Shared Responsibilities

- All three: contribute commits to GitHub, review each other's code, present during demonstration
- Presentation split: Darien (data & logic), Domanick (GUI & validation), Tean(reports & testing)